

LLM based Fault Diagnostic Chatbot Web Application

Name	Sultan Mamud
CC No	5945
Lab	Accelerator Control Section (ACS)
From	02/06/2025
To	02/09/2025
Section Head	Shri Rajesh Kumar Agrawal (Head, Accelerator Controls Section)
Coordinator	Smt. Bhavna Nitin Merh (Head, Software, Networking & Security Lab.)
Organization	RRCAT

Table of Contents

Table of Contents:	2
1. Abstract:	3
2. System Requirements:	3
2.1 Functional Requirements:	3
2.2 Non-Functional Requirements:	3
3. Features:	4
4. Methodology:	5
4.1 Requirement Gathering:	5
4.2 Design:	5
4.3 Implementation:	5
4.3.1 Environment Setup:	5
4.3.2 Backend Development (Flask):	6
4.3.3 Frontend Development:	6
4.3.4 Fine-Tuning the NL2SQL Model (Mistral 7B):	6
4.3.5 RAG Pipeline Setup and Data Ingestion:	7
4.4 Testing:	7
4.5 Deployment:	8
5. Result:	8
6. Challenges:	8
6.1. Model Selection & Optimization:	8
6.2. Training & Fine-Tuning:	9
6.3. Conversational Context Management:	9
6.4. System Integration & Performance:	9
6.5. Security & Offline Operation:	9
7. Future Scope:	9
8. Conclusion:	10
9. References:	10

1. Abstract:

This report presents the design, development, testing, and deployment of the **Indus Faultbook Assistant**, a web-based diagnostic tool for Indus Accelerator Systems. The application integrates state-of-the-art techniques, including **QLoRA fine-tuning of open-source large language models (LLMs)**, **Retrieval-Augmented Generation (RAG)**, and **vector database search**, to deliver accurate, context-aware responses. By interfacing with the existing internal SQL fault logbook, the assistant enables engineers and technicians to query, analyze, and interpret fault records in natural language, reducing reliance on manual SQL queries. The system is designed for secure, offline deployment within the organizational network, ensuring both **operational efficiency** and **data confidentiality**.

2. System Requirements

2.1 Functional Requirements

1. Database Integration & Querying

- The system must connect to the existing flogbook SQL Server database to retrieve fault log records based on user queries.
- It must accept natural language queries regarding faults and incidents, handle follow-up questions using conversational context, and return precise answers from the database.

2. Advanced Search & Data Insights

- The chatbot must allow users to search and filter logs by criteria such as fault ID, date range, and subsystem.
- It must provide natural language summaries and statistical insights (e.g., most frequent faults) instead of only raw records.

3. Response & Logging

- Responses should be displayed in structured formats such as tables or charts where appropriate.
- The system must maintain detailed logs of all user interactions and executed SQL queries for auditing and debugging purposes.

4. Offline Operation

- The entire application, including all models and database operations, must be deployed and run on the internal secure network without requiring internet access.

2.2 Non-Functional Requirements

1. Performance

- Common user queries must be answered within **5–10 seconds**, and complex SQL-based requests must be completed within **15 seconds**.
- The system must support at least **3 concurrent users** without performance degradation.

2. Maintainability & Scalability

- System configurations such as model paths and database connections must be easily updatable through external configuration files.
- The database must be optimized to ensure performance does not degrade as data volume grows.

3. Usability

- The user interface must be intuitive, providing clear prompts and error messages.

3. Features:

The project followed an agile development lifecycle, with continuous iteration through design, implementation, and testing.

Feature	Implementation Details
Intent Classification	An ensemble approach was used for maximum accuracy. A user's query is classified by three models in parallel: a 4-bit quantized Mistral 7B using advanced prompt engineering, a fine-tuned TinyLlama 1B model, and a fine-tuned BERT classifier. A voting and confidence-based logic determines the final intent, which routes the query to the appropriate pipeline.
Natural Language to SQL	For INTENT1_FAULT_SQL, a specialized, fine-tuned version of the Mistral 7B model is used. It takes the user's natural language query and generates a corresponding SQL query. The system then validates the syntax of this query before executing it against the flogbook database.
RAG Implementation	For INTENT2_FAULT_RAG and INTENT3_DOMAIN_RAG, a vector database was populated with historical fault logs and internal technical documents. When a query is received, the system retrieves the most relevant "chunks" of text from this database and provides them to the Mistral 7B model as context to generate a detailed, expert-level answer.
Query Rewriting	To handle follow-up questions, a query rewriting module was implemented. It uses the base Mistral 7B model to analyze the conversation history and rewrite an ambiguous follow-up (e.g., "what about for the other one?") into a complete, standalone question that can be understood by the downstream models.
Conversation Memory	The system maintains conversation memory by storing every turn of the dialogue in a dedicated SQL Server table, linked by a unique session_id. For each new query, the recent history is loaded from the database to provide context for the query rewriter.
Real-Time Streaming	The Flask backend uses a generator function to stream the LLM's response token-by-token. The frontend uses the fetch API to consume these Server-Sent Events (SSE), creating a real-time "typing" effect for the user.
GPU Utilization	The LLMs are loaded in 4-bit precision using the bitsandbytes library, which significantly reduces the VRAM requirement and allows multiple models to run on a single GPU. The application is configured to automatically use the available CUDA device.

4. Methodology

4.1 Requirement Gathering:

The chatbot is intended to assist technicians and engineers in the **Indus Operation Division (IOD)** and **Accelerator Control Section (ACS)**. Currently, users rely on manual SQL queries on the flogbook database, which is time-consuming, requires SQL knowledge, and provides only raw results without deeper insights.

Issues Identified:

- Limited SQL expertise among users.
- Slow and error-prone manual query process.
- Difficulty in analyzing large volumes of log data.
- Lack of summarized insights (frequent faults, subsystem trends).

User Expectations:

- Querying fault data through **natural language** without SQL.
- **Filtering and searching** logs by fault ID, subsystem, and time range.
- **Conversational support** for follow-up queries.
- **Summaries and analytics** beyond raw records.
- **Structured outputs** (tables, charts).
- **Offline availability** within the secure internal network.
- **Audit logging** of interactions and executed queries.

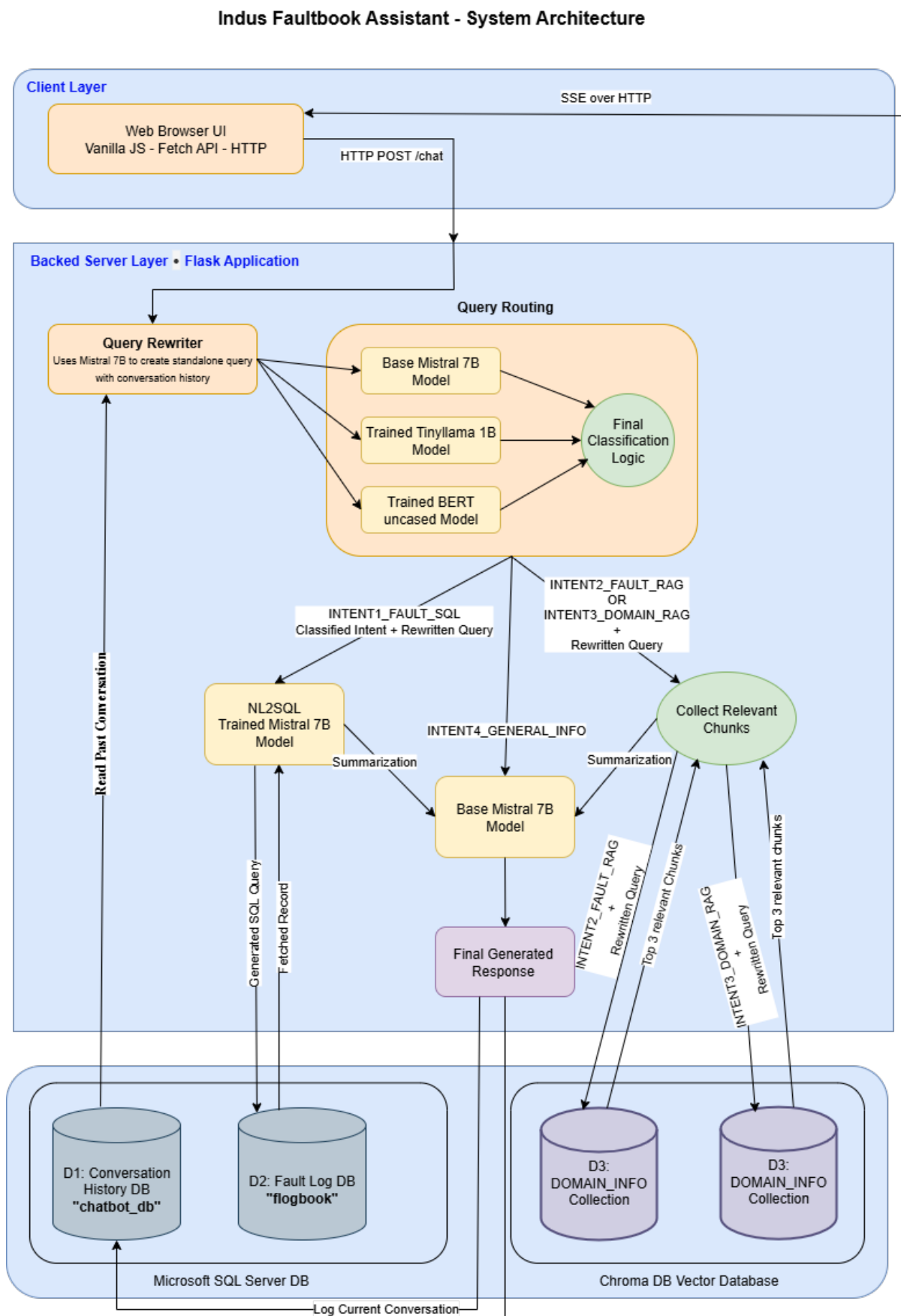
Stakeholders:

- *Technicians* – quick access to fault logs for troubleshooting.
- *Engineers* – analytical summaries and trend analysis.
- *Administrators* – security, offline deployment, and audit compliance.

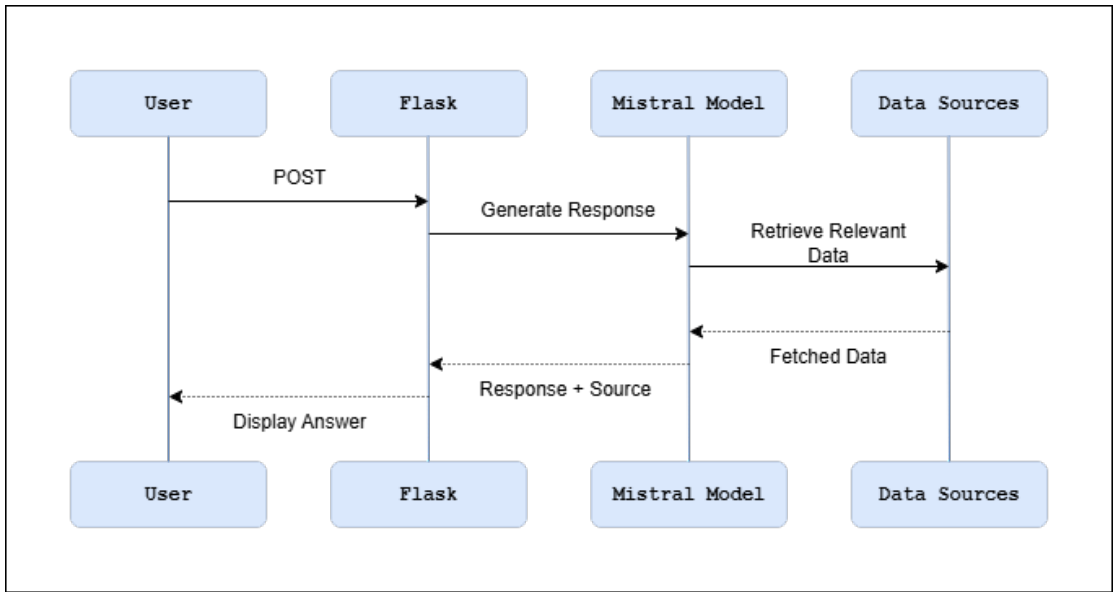
These findings define the **functional** and **non-functional requirements** of the Indus Faultbook Assistant, ensuring it addresses user needs while operating securely within ACS infrastructure.

4.2 Design:

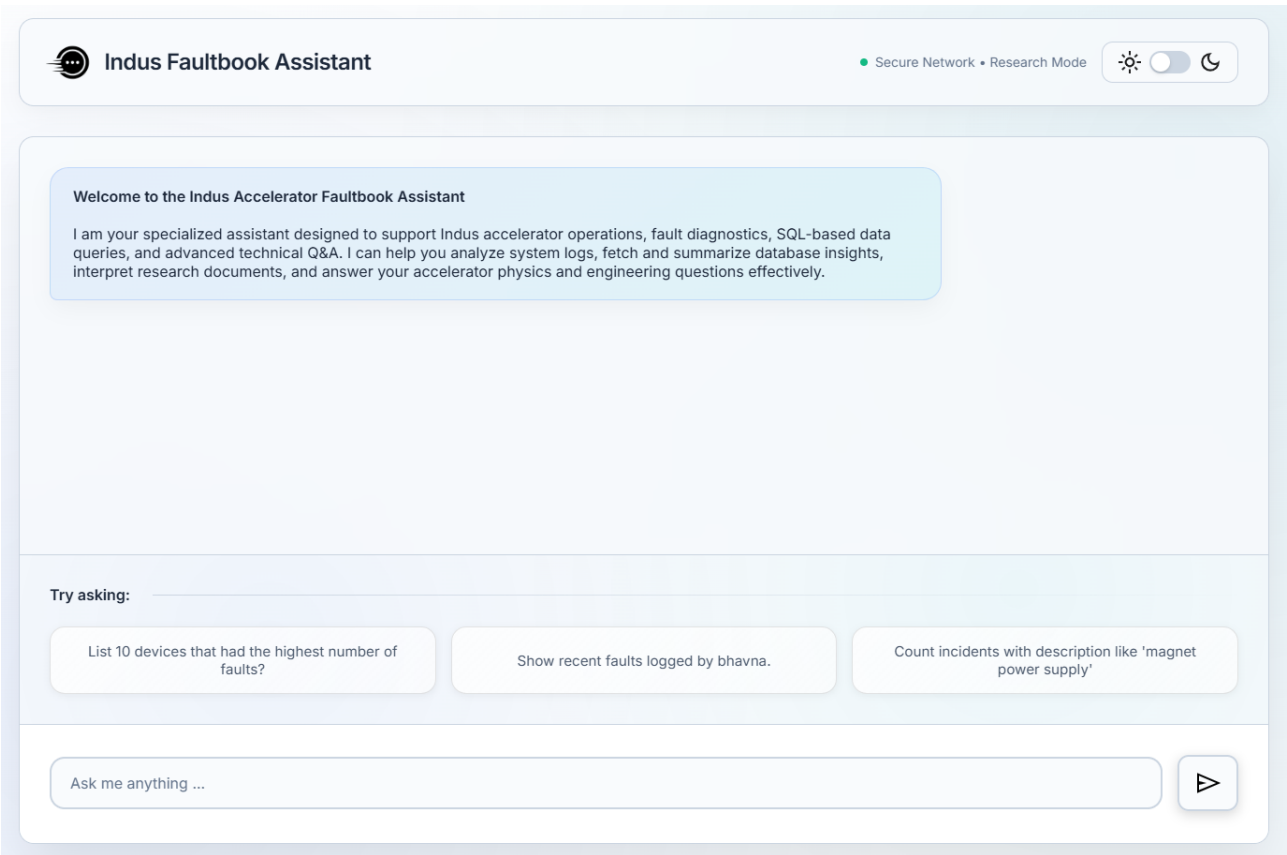
System Architecture Diagram:

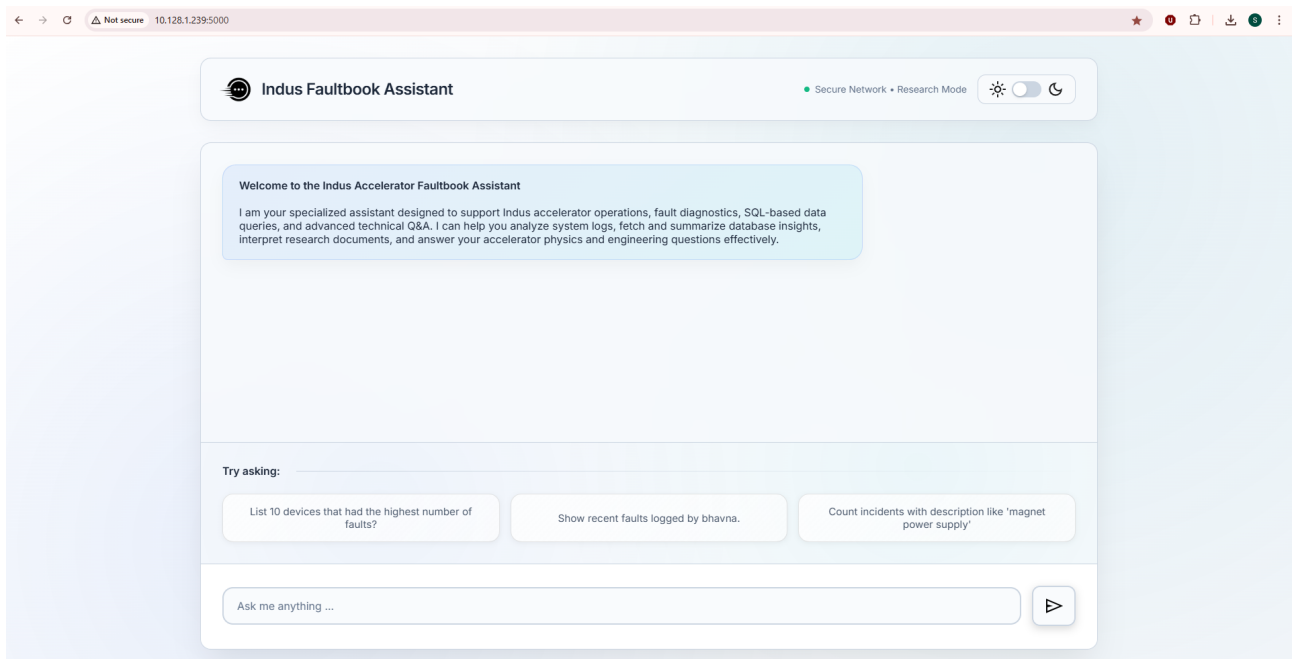


High Level Sequence Diagram:



Frontend Design:





4.3 Implementation

The implementation phase translated the architectural design into a functional application. Development was segmented into distinct modules: the backend server, the frontend user interface, the core AI pipeline, and the specialized fine-tuning and data ingestion processes.

4.3.1 Environment Setup

The project was developed in a Python 3.10 environment. The core dependencies were managed via a requirements.txt file and included:

- **Web Framework:** Flask for the backend server and API endpoints.
- **AI & ML Libraries:** torch, transformers, peft, and bitsandbytes for loading, quantizing, and running the LLMs.
- **Database:** SQLAlchemy and pyodbc for connecting to the SQL Server database.
- **Vector Store:** langchain-chroma for creating and querying the local vector database.
- All models were configured to run on a local NVIDIA GPU, leveraging CUDA for accelerated performance.

4.3.2 Backend Development (Flask)

The backend was built as a monolithic Flask application responsible for all core logic.

- **Routing:** Endpoints were created for serving the main HTML page (/), handling the streaming chat (/stream_chat), and managing user sessions (/start_session, /end_session).
- **Streaming Logic:** The /stream_chat endpoint was implemented using a Python generator function. This allowed the application to send Server-Sent Events (SSE) to the frontend, streaming the LLM's response token-by-token for a real-time experience.
- **Database Integration:** SQLAlchemy was used to create a connection engine to the SQL Server. Functions were implemented to log every conversation turn (user prompt, rewritten prompt, intent, final response, etc.) and to retrieve conversation history for the query rewriter.

4.3.3 Frontend Development

The user interface was built with standard web technologies to ensure simplicity and performance.

- **Structure & Styling:** The UI was structured with **HTML** and styled with custom **CSS** to create a clean, modern chat interface, including a theme-toggle for light and dark modes.
- **Client-Side Logic: Vanilla JavaScript** was used to handle all user interactions. The `sendMessage` function captures user input and uses the `fetch` API to communicate with the backend's `/stream_chat` endpoint. It is designed to parse the incoming SSE stream, update the UI with response chunks, and handle special events like pipeline status updates and final SQL query displays.

4.3.4 Fine-Tuning the NL2SQL Model (Mistral 7B)

A critical part of the implementation was fine-tuning the Mistral 7B model to accurately translate natural language questions into SQL queries for the flogbook database. This was achieved using the QLoRA technique.

Step 1: Dataset Preparation

A high-quality dataset is the most crucial element for successful fine-tuning. A JSON or CSV file was created containing several hundred examples of question-query pairs. Each entry followed a structured format that the model could be trained on.

- Example Data Point:

```
{
  "text": "<s>[INST] What is the total number of faults recorded? [/INST] SELECT COUNT(*) FROM
fault_bookv3; </s>"
},
{
  "text": "<s>[INST] Show me the fault description for fault id 6771 [/INST] SELECT fault_description FROM
fault_bookv3 WHERE fault_id = 6771; </s>"
}
```

The [INST] tags are the instruction format Mistral was trained on, which helps the model understand the task.

Step 2: Model Configuration (QLoRA)

The base Mistral 7B model was loaded using the transformers library. To make training efficient, a `BitsAndBytesConfig` was created to load the model in 4-bit precision. A `LoraConfig` from the `peft` library was then defined to specify which parts of the model to train (typically the attention layers) and to set the parameters for the low-rank adapters.

Step 3: The Training Process

The `SFTTrainer` (Supervised Fine-tuning Trainer) from the `trl` library was used to orchestrate the training.

1. The trainer was initialized with the **4-bit quantized** base model, the prepared dataset, and the LoRA configuration.
2. Training arguments were defined, including the number of epochs, learning rate, and batch size.
3. The `trainer.train()` method was called, which started the fine-tuning process. The trainer handled the process of feeding data to the model and updating the small set of trainable LoRA weights.
4. After training completed, the resulting adapter weights were saved to a separate directory.

Step 4: Integration

In the main Flask application, the NL2SQL model was loaded by first loading the 4-bit base Mistral 7B model and then using PEFT's `PeftModel.from_pretrained()` method to attach the newly trained LoRA adapter weights. This combined model was then used exclusively for the NL2SQL pipeline.

4.3.5 RAG Pipeline Setup and Data Ingestion

To enable the chatbot to answer questions beyond the structured SQL data, a RAG pipeline was implemented to create a searchable knowledge base from unstructured documents and the fault logbook itself.

Step 1: Data Ingestion and Processing

Two primary sources of data were used for the knowledge base:

- **Unstructured Documents:** Technical manuals, PDFs, and JSON files related to accelerator physics and database schemas were loaded using the **LangChain** document loaders.
- **Fault Logbook Data:** The entire **fault_bookv3** table was fetched from SQL Server into a Pandas DataFrame. This data was then cleaned and preprocessed—normalizing dates, cleaning duration fields, and formatting each row into a human-readable text document.

Step 2: Text Chunking

All loaded documents were split into smaller, semantically coherent chunks using the **RecursiveCharacterTextSplitter**. This is a crucial step to ensure that the embeddings capture the specific context of each piece of information and that the retrieved context is concise and relevant to the user's query.

Step 3: Embedding and Storage

The **HuggingFaceEmbeddings** library was used to load a local **all-MiniLM-L6-v2** sentence transformer model. This model was used to convert each text chunk into a numerical vector (embedding). These embeddings were then stored in a persistent **ChromaDB** vector store on the local disk. Separate collections were created for **FAULT_INFO** and **DOMAIN_INFO** to keep the knowledge bases distinct.

Step 4: Retrieval

At runtime, when a RAG-classified query is received, the **retrieve_from_collection** function is called. It first converts the user's query into an embedding using the same **all-MiniLM-L6-v2** model. It then performs a similarity search against the ChromaDB collection to find the top k most relevant text chunks. This retrieved context is then passed to the main Mistral 7B LLM to generate the final, informed answer.

4.4 Testing

The testing phase was a multi-faceted process designed to ensure the application's reliability, accuracy, and performance. The strategy was divided into three key stages:

- **Unit & Integration Testing:** Each module was tested individually before being integrated into the main application. This included verifying the accuracy of the **intent classification** ensemble, validating the syntax of the generated SQL from the **NL2SQL pipeline**, and confirming the relevance of documents retrieved by the **RAG pipeline**. Integration tests then ensured that the data flowed correctly through the entire architecture, from query rewriting to the final response generation.
- **Performance Testing:** The system was benchmarked against the non-functional requirements. Response times for various query types were measured to ensure they met the **5-10 second** target for common requests and the **15-second limit** for complex queries.

4.5 Deployment

The **Indus Faultbook Assistant** will be deployed within the internal secure network of the Accelerator Control Section (ACS) to ensure reliability, data integrity, and compliance with organizational security policies. The deployment strategy will be designed to support offline operation, ease of maintenance, and scalability for future requirements.

5. Result:

System Configuration:

Processor	Intel i7 13 th Gen
RAM	32GB
Storage	1TB
GPU	NVIDIA RTX 3060 with 12GB VRAM

Result on the above system:

Matrix	Result
Average Inference Speed	25 tokens/sec
GPU Utilization	11GB out of 12GB
CPU Utilization (During inferencing)	24-34%
System RAM Utilization	Additional 2.5GB compared to idle state

6. Challenges

The development and deployment of the **Indus Faultbook Assistant** presents several challenges that need to be carefully addressed for the system to operate effectively in an offline, mission-critical environment.

6.1. Model Selection & Optimization

- **Choosing the Right LLM:** Selecting a suitable language model for different tasks (general conversational pipeline, NL2SQL generation, fault log summarization) is critical. Larger models offer better accuracy but demand more computational resources, which may not be feasible in an offline setup.
- **Selecting an Embedding Model:** Identifying an embedding model that balances semantic accuracy with efficiency is essential for similarity search and context retrieval.
- **Choosing the Vector Database:** Since the system must operate offline, selecting a vector database that supports efficient search, scalability, and easy integration with SQL Server is a challenge.

6.2. Training & Fine-Tuning

- **Training NL2SQL Pipeline:** Converting natural language queries into optimized SQL queries requires model fine-tuning with domain-specific examples. Building and validating such a pipeline is complex.
- **Preparing Domain-Specific Datasets:** Creating high-quality datasets for NL2SQL training, intent classification, and fault summarization involves cleaning, labeling, and balancing fault log records, which is resource-intensive.
- **Intent Classification:** Accurately identifying user intents (e.g., fault search, statistics request, data summary) is necessary for query routing and reducing ambiguity.

6.3. Conversational Context Management

- **Maintaining Conversation History:** Ensuring that the system correctly handles follow-up questions and multi-turn conversations without losing context is a key challenge, especially under limited memory constraints.

- **Balancing Accuracy vs. Efficiency:** Conversation memory storage must be optimized to avoid excessive resource consumption while still providing meaningful continuity.

6.4. System Integration & Performance

- **Integration with SQL Server:** The model must generate syntactically correct and performant SQL queries compatible with the existing flogbook schema.
- **Query Execution Efficiency:** Some user queries may involve large datasets, and ensuring responses are returned within the defined performance limits is challenging.
- **Concurrent Usage:** Supporting multiple engineers simultaneously without degrading performance requires careful optimization of both the LLM pipeline and database queries.

6.5. Security & Offline Operation

- **Offline Deployment:** Since the system cannot rely on cloud APIs, all models, embeddings, and databases must be hosted internally, increasing the burden of resource management.
- **Data Privacy:** Fault log records contain sensitive operational data, making it essential to ensure strict access control, secure logging, and prevention of data leakage.

7. Future Scope

1. **Real-Time Database Synchronization:** Ensuring seamless and up-to-date data exchange between the chatbot and live databases for optimal performance.
2. **Multi-Modal Input Support:** Expanding input capabilities to accommodate various formats, such as text, voice, images, and video, for a more intuitive and interactive user experience.
3. **Advanced Reporting & Export Options:** Allowing users to export chatbot responses and comprehensive reports in formats like PDF, CSV, or Excel, enhancing data analysis and presentation capabilities.
4. **Secure User Authentication & Access Control:** Implementing robust authentication mechanisms and fine-grained access control to maintain user privacy and restrict unauthorized access to sensitive information.
5. **Scalability through Large Language Model (vLLM) and FastAPI Integration:** Building a scalable and high-performance chatbot that can efficiently handle a large volume of requests by leveraging the power of modern inferencing engines (eg: vLLM) and FastAPI framework.
6. **Platform Portability:** Ensuring cross-platform compatibility by adapting the chatbot to run seamlessly on Linux, Windows, and Docker environments, maximizing its reach and usability across various platforms.

8. Conclusion

The **Indus Faultbook Assistant** successfully addresses the limitations of manual SQL-based fault log retrieval by introducing a conversational, intelligent interface tailored for the Indus Operation Division (IOD) and Accelerator Control Section (ACS). By integrating advanced techniques such as **QLoRA fine-tuning**, **Retrieval-Augmented Generation (RAG)**, and **vector database search**, the system enables engineers and technicians to obtain accurate, context-aware insights directly from the existing fault logbook.

The deployment of this solution within the secure, offline environment ensures both **operational reliability** and **data confidentiality**, aligning with organizational requirements. In addition to simplifying fault diagnostics, the system lays the foundation for **scalable enhancements** such as predictive fault analysis, subsystem health

monitoring, and richer visualization tools. Overall, the assistant demonstrates how state-of-the-art natural language processing can be effectively adapted to mission-critical, domain-specific applications.

9. References

1. Touvron, H. et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv preprint, 2023.
2. Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint, 2020.
3. Microsoft SQL Server Documentation.
4. Uber Engineering. *QueryGPT: Natural Language Interface for SQL Databases*. Uber Engineering Blog, 2023.
5. RRCAT Internal Documentation – “flogbook database” and “Infonet”